ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/4123211

Analysis of microbenchmarks for performance tuning of clusters

Conference Paper · October 2004

DOI: 10.1109/CLUSTR.2004.1392636 · Source: IEEE Xplore

CITATIONS
38

READS
228

2 authors:



Matthew J. Sottile
Lawrence Livermore National Laboratory
84 PUBLICATIONS **843** CITATIONS
SEE PROFILE



Ronald G Minnich
Google Inc.
81 PUBLICATIONS **1,385** CITATIONS
SEE PROFILE

All content following this page was uploaded by Matthew J. Sottile on 25 March 2015.

The user has requested enhancement of the downloaded file.

Analysis of microbenchmarks for performance tuning of clusters

Matthew Sottile and Ronald Minnich
Los Alamos National Laboratory
Advanced Computing Laboratory, MS-B287
Los Alamos, NM 87545 USA
{matt,rminnich}@lanl.gov

Abstract— **Microbenchmarks, i.e. very small computational kernels, have become commonly used for quantitative measures of node performance in clusters. For example, a commonly used benchmark measures the amount of time required to perform a fixed quantum of work. Unfortunately, this benchmark is one of many that violate well known rules from sampling theory, leading to erroneous, contradictory or misleading results. At a minimum, these types of benchmarks can not be used to identify time-based activities that may interfere with and hence limit application performance. Our original and primary goal remains to identify noise in the system due to periodic activities that are not part of user application code. In this paper, we discuss why the 'fixed quantum of work' benchmark provides data that is of limited use for analysis; and we show code for, discuss, and analyze results from a microbenchmark which follows good rules of sampling hygiene, and hence provides useful data for analysis.**

I. Introduction and Background

Cluster-based supercomputers continue to be a popular high-performance computing platform, and have recently broken into the large-scale supercomputing realm previously occupied by custom and traditional parallel architectures. The most immature component of clusters relative to traditional HPC platforms is the compute node operating system architecture. Until recently, compute nodes were no different than workstations in both hardware and software configuration. With minimal modification to Linux itself, systems such as BProc [4] and OpenMOSIX [2] are able to eliminate much of the useless on-node software. This second generation of cluster node software architectures eliminates obvious sources of perturbation: daemons that require resources, unnecessary kernel code (such as unused modules), and excessive hardware. The primary motivation for eliminating hardware is one of reliability, but a secondary benefit is that it also eliminates potential sources of noise due to hardware interrupts.

Noise due to hardware interrupts or other interference can lead to a program running in a longer time. We show a simple situation in Figure 1.

In the first case, the program is fine – it starts, runs with the whole CPU, and ends at the expected time, T_{done}. In the second case, some interrupt has occurred, caused by either hardware or software, and the program completion time is delayed.

Los Alamos National Laboratory is operated by the University of California for the National Nuclear Security Administration of the United States Department of Energy under contract W-7405-ENG-36, LA-UR No. 04-3037.

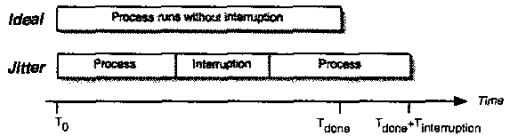


Fig. 1. Interference and how it can slow up an application

Why does the interference even matter? Consider a 1024 node cluster application running 1024 totally independent tasks, each one taking a minute. Suppose some interruption occurs on one node that delays that single node by 50 milliseconds. The result will be that the application runs as though there are only 1023 nodes in the cluster, not 1024. The application scaling is ruined – the cluster only ran the one minute job as fast as 1023 nodes, not 1024 nodes. The measure of success on clusters is a very fine one – single node perturbations can have a severe impact on the performance of a parallel application.

If the application's tasks are not totally independent, but instead need to communicate at regular intervals, interference can have a huge impact. Consider the case of 1024 tasks which communicate at 1 millisecond intervals, which is not an unrealistic scenario nowadays in scientific workloads. For this case, interference that results in delays of a few microseconds can impact the entire application by causing many nodes to sit idle waiting for the perturbed nodes to catch up. This is why careful tuning and elimination of single node perturbations is a vital step in tuning an entire cluster.

Scientific applications have very special characteristics relative to the suite of applications a general operating system is designed to support. First, consider the simple uniprocessor case. A simple application such as a 2D grid-based solver requires long periods of sustained computation on a fixed amount of memory. In a well written implementation, locality can be leveraged to use caching or vector hardware to perform long sequences of simple operations on many data elements very quickly. Caching in particular requires that in order for these long sequences to occur quickly, the cache remains consistent and does not need to be read again. Similarly, the processor should not switch contexts and give cycles to other processes. If memory, TLB, cache, or any other time critical component is corrupted by another process, there is overhead

involved in reloading elements so that the instruction sequence can complete. If a process is preempted for other activities, there is both time spent by the other activities and that for context switching. Both are high overhead and increase the amount of time a given amount of work takes to complete.

Locating sources of interference can be difficult. Once the obvious sources of interference such as daemons are removed and compute nodes are turned into essentially dedicated computational resources, the really hard work begins. Many portions of the Linux kernel have sufficient performance for general use, but tuning may be required to minimize the impact of portions such as the scheduler or software interrupts on both compute-heavy and globally synchronous workloads characteristic of many scientific applications. Furthermore, in distributed systems such as BProc, there are infrequent but periodic activities that the operating system performs to ensure that the overall cluster remains functional over time.

A key activity in locating interference is the quality of the measurement tools. Over the last 10 years researchers have developed several tools to measure interference, usually as a set of microbenchmarks [3] [5] [9]. A popular microbenchmark is the 'fixed work quantum' (FWQ) benchmark, in which a program reads the time, performs a fixed amount of work, reads the time again, and saves the delta. The FWQ has shown its value several times for determining that interference exists.

As it happens, the FWQ benchmark has limitations. It fails to take into account important fundamentals of sampling theory [7] [6], and it can easily be fooled. For example, a common analysis technique is to look at the variance of the time it takes to perform the work. On a saturated processor the variance measure will show no interference at all – it is not possible to distinguish between a very slow processor and one with no interference. This is discussed in detail in Section IV.

Finally, and most importantly, the data output of FWQ is unusable with many popular techniques, such as the Fourier transform and periodogram analysis [1], due to the fact that samples are taken every quantum of work. Reconstructing the amount of work performed as a function of time is very difficult, and requires assumptions about the distribution of work over the sampled time interval that are unverifiable. As such, it is difficult to make statements about activities in the system in either the time or frequency domain given the lack of temporal data.

In this paper we discuss the flaws of FWQ benchmark, and provide a new microbenchmark, 'fixed time quantum', or FTQ, which yields far more useful results. We then show some of the results from FTQ and how it can be used for frequency domain analysis of interference. Since many of the causes of interference are periodic in nature, frequency domain analysis can point to a root cause of interference.

This work is motivated by the following condition. We examine the amount of work a program is able to do by periodically sampling the work it has achieved. The clock we use is accurate enough to characterize many different phenomena in both the operating system and the hardware. The microbenchmark we have developed guarantees that we adhere to good practice in that our sample intervals are fixed in duration, occurring at periodic intervals, and that the samples are taken on well-defined time boundaries. We can measure both utilization as percent-of-peak and interference. Preemption of processes and inefficient cache usage cause a lower utilization of a processor for a given workload. Our goal is to increase (not necessarily optimally) the utilization achieved by applications by tuning the operating system to reduce high impact interference and overhead.

In this paper we will present our measurement and benchmarking methodology, experimental setup, data analysis, and preliminary conclusions. The benchmark this work is based on is detailed in Section II. We provide a detailed description of the experimental setup (both software and hardware) and process (how and when measurements were taken) in Section III. Finally, Section IV provides a basic overview of the analysis techniques we have and are continuing to investigate to identify features in the raw benchmark output that are related to noise sources. We will also show some preliminary data resulting from this study that illustrates why this work is relevant.

II. Microbenchmarks

In this section, we begin with a discussion of timers that are used for reporting timing data for microbenchmarks. We then discuss an existing, widely used microbenchmark based on timing fixed workloads and the flaws inherent in its design. Given these flaws, we present a microbenchmark that overcomes these flaws by measuring work completed in a fixed quantum of time.

A. A brief discussion of time

Before we discuss microbenchmarks that use timers, we need to discuss timers. What kind of timers have been used for these microbenchmarks? The most common timer is `gettimeofday()`.

Consider an inherent limitation of software timers. If t units of real time elapse from some measurement t_0 and its successor t_1, we have to assume that there is some small error ϵ that forces

$$t = (t_1 - t_0) + \epsilon \tag{1}$$

If we wish to derive information about the distribution of the interval lengths in a series of time samples, we must minimize or eliminate ϵ by carefully choosing the `readtimer()` implementation. Our benchmark code currently uses a processor tick-based timing function, which is known to be accurate to within a sub-nanosecond resolution. Many benchmarks that are similar to this one use the more common `gettimeofday()` function from C. That timer has a higher amount of variance and a lower precision than counting raw processor ticks, making it difficult to assess the effect of fine-grain hardware or operating system activities.

Another consideration is the Nyquist frequency and sampling theorem. In sampling-based systems, the Nyquist frequency is twice the highest frequency which can sensibly be measured for a given sampling rate. The sampling theorem states that reconstruction of a band-limited signal (i.e., the

```
for i=1 to n do
  timestamp = readtimer()
  timeseries[i] = timestamp
  for (j=1;j<workcount;j++)
    operation
  end
end
```

Fig. 2. Pseudocode for the Fixed Work Quantum (FWQ) microbenchmark.

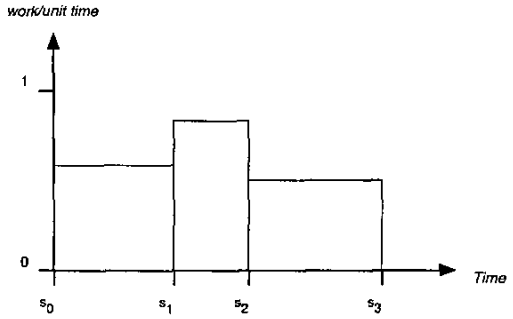


Fig. 3. An example of three samples of the FWQ benchmark.

power spectrum for frequencies $\omega > B$ is zero), one must sample at a rate $\omega_{sample} \geq 2B$. If $\omega_{sample} = \omega_{Nyquist} = 2B$, the signal is said to be *Nyquist sampled.* In a system with a 2GHz clock, to fully reconstruct a signal band-limited to $0 \leq \omega \leq 200Hz$, sampling at a Nyquist frequency of 400Hz is achievable with the high resolution timers. If a microbenchmark uses the `gettimeofday()` clock which runs at 100Hz, then we can only really talk about signals of 50Hz or less. High resolution timers driven by the processor clock can support sampling with a Nyquist frequency of nearly 1GHz. As such, they allow microbenchmarks to detect perturbations occurring at realistic frequencies of 100-200Hz, or interrupts occurring at several KHz.

Most modern processors provide some mechanism to measure processor ticks for high resolution timing applications. The Pentium, PowerPC, AMD Athlon, Sparc, Alpha, and MIPS processor families all have provided cycle-based counters for performance analysis for many years. In our experiments, we use the Pentium Time Stamp Counter (TSC). For robust performance analysis and measurement, there is no excuse to use poor precision and accuracy counters such as `gettimeofday()`. One cannot assume that the underlying implementation uses high-precision processor timers, so it should simply be avoided. Furthermore, many timers provided for libraries such as MPI (e.g., `MPI_Wtime()`) use `gettimeofday()`. The moral to remember is that understanding the capabilities and limitations of your timers is vital to reaching meaningful and sound conclusions from time measurements.

B. The Fixed Work Quantum (FWQ) microbenchmark

The FWQ benchmark is used extensively in the community. It measures the amount of time required to perform a fixed amount of work, where each unit of work is either a memory reference, floating point, or integer arithmetic operation. The code can be summarized in pseudocode as shown in Fig. 2.

Although trivial at first glance, there are features of this that must be understood to properly interpret data. First, to preserve the raw data, we must store it in memory. Given that n is assumed to be significantly larger than a cache line size, we will see some caching effects when the index addresses a location not located in cache close to the processor. Modern systems with predictive prefetching and other hardware optimizations will be able to take advantage of the simplicity of the workload to hide these effects, but in general they will be present to some degree.

How well does this benchmark work? For high performance nodes, with complex sources of interference, not well at all. It will provide some indication of interference. We have seen that it is just as likely to provide no indication when interference is maximized and uniformly distributed over the sampling period.

Finally, there may be a huge amount of interference, which the benchmark might reveal in some gross sense, but the data will not reveal the details. This is true regardless of the quality of the timer used. The more interference there is in the system, the more interference in the benchmark and the more variance in the actual timestamps. The sample intervals will thus be of variable width and variable starting point. They are useless for any type of signal processing. This is illustrated in Fig. 3 where three samples are taken of a fixed workload. The fixed workload corresponds to the area of the rectangle over the time period required for the fixed amount of work to complete. The FWQ benchmark only dictates this constant amount of work with no information regarding the distribution of this work over fixed width time intervals.

The data, rather than revealing variance, is a mirror of the variance of the system. It is as though the system were sampling noise instead of information. Here we see a fundamental flaw in the fixed work quantum benchmark.

C. The Fixed Time Quantum (FTQ) microbenchmark

In response to the shortcomings of the FWQ benchmark we developed the fixed time quantum (FTQ) benchmark. FTQ is well-behaved in the way it samples and hence can be used for frequency- and time-domain analysis. We show the inner loop that performs the sampling in Fig. 4.

The basic operation is simple. The loop will count work done until it reaches a fixed end-point in time. It then records the starting point of the loop and the amount of work that was done.

This microbenchmark measures *work per unit of time*, with a fixed unit of time, ending at a periodic interval. It does everything right that the other microbenchmark does wrong. The data from this benchmark can hence be used with such tools as the Fourier transform and time series analysis methods. We present results in the following sections.

```
// sample interval = 2^21 ticks
terminal = 1 << 21;

while (!endloop) {
  last = readtimer()
  endinterval = (last + terminal)
                & (~(terminal-1));
  for(now = last, count = 0;
      now  < endinterval;) {
    count++;
    now = readtimer();
  }

  totalcount[(done)*2] = last;
  totalcount[(done)*2+1] = count;
  done++;
  if (done > dototalcount)
    endloop = true;
}
```

Fig. 4. Pseudocode for the Fixed Time Quantum (FTQ) microbenchmark.

The difference between measuring work per unit time versus time per unit work is the key to the analysis being able to accurately identify periodic features in the data. Periodicity of interference is measured in cycles per unit time, most commonly cycles per second (Hz). Periodicity in the FWQ benchmark data is cycles per unit work, which is very difficult to correlate with activities that are periodic in time. We briefly looked at methods for interpolating time per unit work data into work per unit time, but gross assumptions have to be made about the distribution of work over the measured time period for each sample. Our desire to use tested techniques from the signal processing and spectral analysis fields led us to find a way to sample the system in a domain that those methods are well defined over: fixed time interval sampling.

III. Experimental Methodology

Accurate benchmarking is a difficult process, and frequently conclusions that are reached are not as strong as they could be due to under-specified and poorly controlled experimental conditions. Our primary goal in the experiments here, other than gathering and analyzing valid data, is ensuring that third parties be able to reproduce the results. As such, this section details the process we followed in setting up and executing the benchmarking experiment in hopes that we both ensure reliability and eliminate as many variables as possible from the activity. We also point out potential variables that are not currently controlled during the experiment.

A. Hardware and software configuration

The objective of the experiment is as follows. Given a compute node, evaluate the behavior of a fixed benchmark in the presence of different operating system environments. Identical binaries are used regardless of the operating system configuration. The hardware remains constant - no devices are to be removed or added. In cases where devices are unnecessary, such as network-based booting of nodes, the drives required for other operating systems were left in place and enabled. A later experiment will then test whether or not unused devices being removed has any influence on the outcome of the benchmark results. In this paper, the data we present was gathered on a single operating system configuration from an existing BProc-based cluster node. We are now at the point where the benchmark is tuned and understood to the point where we can begin to do cross-operating system comparisons.

The benchmark code itself is described in detail in Section II. Before running the benchmark, the compute nodes are rebooted and the benchmark is immediately executed on a fresh node. The code will be executed for an identical sampling period, and results will be returned to the frontend when the code has completed. To eliminate artifacts in the data due to poor timer quality and accuracy, we chose to measure durations in terms of processor ticks, using the Pentium TSC. The TSC is a cycle counter, implemented as a register, and is hence accurate to within several hundred picoseconds. This accuracy is in stark contrast to the time of day clock, which is only accurate to several tens of milliseconds.

The kernel on the compute nodes is the Linux SMP kernel version 2.4.25 patched to include BProc. The compute node is composed of a dual processor motherboard with 2.20Ghz Intel Xeon CPUs.

Another variable to consider is the actual code generated by the compiler versus that in the benchmark C code. Careful use of volatiles can ensure that no optimization occurs in critical segments. Also, for each run, optimization settings must be the same. All of the code discussed in this paper was compiled with all optimization options turned off.

B. Processor scheduling issues

The hardware configuration used in this experiment is an unmodified node from a moderate-scale (128 node) cluster. As such, the node was configured with both CPUs active on boot. Without the ability to set processor affinity for specific processes, it currently is not possible to guarantee that for a given run of the benchmark the execution occurs completely on a single processor. If the code is scheduled to the other processor during execution, we expect to see a performance hit due to cache effects, internal scheduling within the processor (instruction level), and other related issues. We hope that instruction level issues within the CPU occur fast enough to be beyond the granularity of the timer we are using, but the current work here does not quantify or take this into account. We also would like to run similar experiments with the second CPU disabled at boot time to explore the effect of multiple processors.

IV. Analysis

The analysis phase of this investigation is performed with raw data from the FTQ microbenchmark as described earlier. Qualitative analysis of the data makes it clear that simple

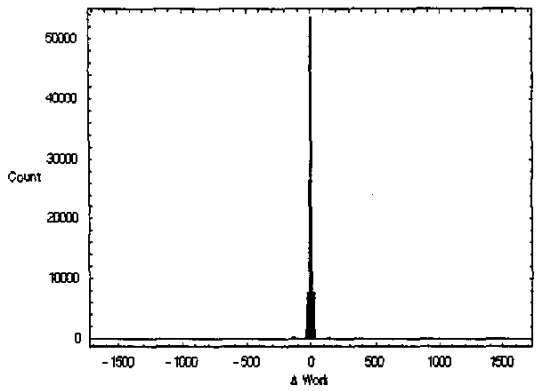


Fig. 5. Histogram of first-order differenced FTQ data.

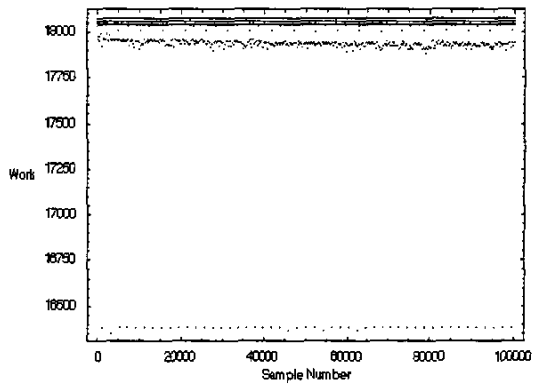


Fig. 6. Plot of raw data for the FTQ benchmark.

visualizations (such as scatterplots) reveal structure and pattern in the data, versus what could appear to be highly random or nearly featureless results. Unfortunately, qualitative techniques are only able to state that there are in fact features in the data that are of interest. The goal is to then quantitatively identify these features by any of a wide variety of techniques.

The first techniques we will discuss are basic statistical methods for analyzing time-series data. These are necessary to answer important questions that impact the choice of more sophisticated methods, such as whether or not the series are stationary or if correlations can be found in the series at different time scales. Calculation of metrics such as the mean and variance are necessary to understand the distribution of measurements. Unfortunately, basic statistics and plots (such as histograms) have an intrinsic problem due to discarding of temporal relationships in the raw datapoints. Two very different time series (one with periodicity, one without) can yield identical sample statistics (mean, variance) and histogram plots. As such, we have and continue to explore more sophisticated techniques that take this into account. To illustrate the little amount of information contained in simple techniques such as histogram analysis, consider Fig. 5 showing a histogram of the first-order differenced FTQ data discussed in detail later. As we can see, most of the data indicates no perturbations between samples, with the majority of samples containing only a small perturbation, and very few outliers. Furthermore, as we will see later, this plot contains no indication of the rich structure of the data in the frequency domain.

Considering that kernel bookkeeping, system heartbeat daemons, and hardware interrupt timers are all periodic in nature, we have been exploring the use of spectral methods to extract periodic signals in the data. This work is still in the exploratory stage and no concrete algorithms have been created that are able to, given a raw benchmark output time-series, provide a set of frequency components that represent periodic interference due to software and hardware.

A. Results

Fig. 6 presents the raw data produced using the FTQ benchmark. The data contains a clear peak-workload at 18077 units of work, with a number of samples that performed less units of work in the fixed sampling interval. The samples are predominantly near or at the peak workload, as the mean of the data is 18072.6 units of work, with a standard deviation of 39.16 units over 100000 samples. Many of the non-peak datapoints show an obvious periodicity, and this motivates our exploration of spectral methods next.

B. Identifying periodicity

Our original and primary goal remains to identify noise in the system due to periodic activities that are not part of user application code. Theoreticians and practitioners in time series analysis have attacked this problem for many years for problems ranging from population dynamics to sunspot cycles in solar data. Although their techniques are far from perfect (consider the complexity of stock market or climate data), for simple systems such as ours, they are quite capable. The basis of our current work is examining features in the frequency spectra of the sampled data using the discrete Fourier transform. A deep explanation of this technique is outside the context of this paper, but the main idea behind it is that most signals can be approximated by composing a set of sinusoidal signals with varying frequencies and magnitudes corresponding to the periodic features of the original signal. More information can be found in [7] and [6].

Consider the sequence of samples shown in Fig. 6. For ease of analysis, first order differencing of the data can produce a zero-mean stationary sample sequence $\hat{S}$ from the original sequence of N samples without loss of frequency-domain information by defining:

$$\hat{s_i} \in \hat{S} = s_i - s_{i-1}\ ,\ i = 2\ ...\ N\ ,\ s_i, s_{i-1} \in S \qquad (2)$$

The resulting sequence $\hat{S}$ is shown in Fig. 7. Using the Mathematica `Fourier[]` function, we produced the spectrum shown in Fig. 8 based on 2000 consecutive samples.

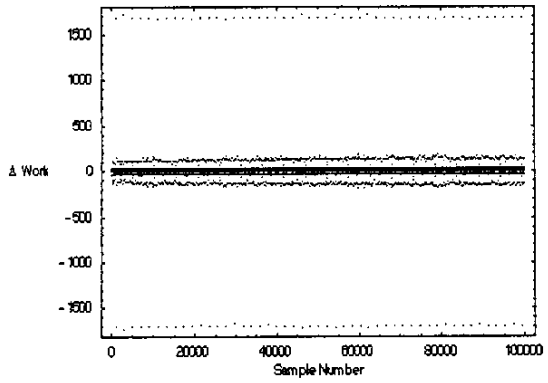


Fig. 7. Plot of data for the FTQ benchmark after first-order differencing.

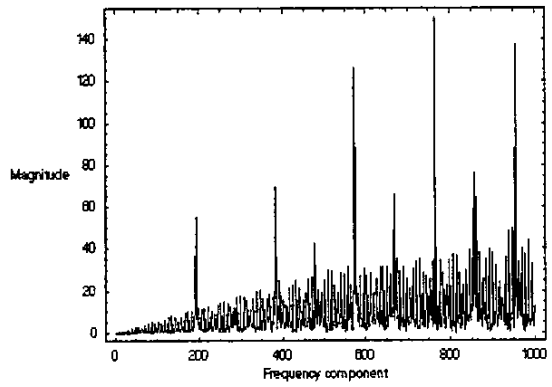


Fig. 8. Plot of the output from Mathematica `Fourier[]` function for 2000 consecutive samples.

As we can plainly see, the frequency spectrum contains a small number of obvious dominant frequency components. We are currently investigating how to interpret these periodic sources of noise in the data and correlate them back to hardware or operating system activities. Once we do this work, we will be able to relate the magnitude of frequency components in this graph to their impact on the application, for example we should be able to state that a frequency component that is only a few dB above the noise floor will have little impact, whereas one that is 20dB over the noise floor may have larger impact. For the first time, we can sample a system, process it using signal processing techniques, and draw conclusions directly from the data.

C. Statistical ambiguity

We have already stated that basic statistics, such as the mean and variance, can be misleading or devoid of information when the source signal does in fact contain information of interest. Furthermore, visualizations such as histograms that are commonly used to qualitatively analyze benchmark data are in fact a very poor tool for identifying correlations between samples due to periodicity and temporal trends.

The use of the variance and mean of a time-series of benchmark data is common, but one must be very careful not to draw complex conclusions about the source data from these simple metrics. Consider a situation where a benchmark is run twice, once on a system with no noise, and once on a system with constant noise (i.e., a constant perturbation uniformly distributed over the time-series). Let δ be the perturbation that interferes with each sample, $S = (s_1, s_2, \cdots)$ be the sample without noise, and $\hat{S} = (\hat{s}_1, \hat{s}_2, \cdots)$ be that with noise. Since the noise is uniformly distributed over the sample sequence, we can let $\hat{s}_i = s_i - \delta$. Assuming the length of S and $\hat{S}$ is N, their means are defined as

$$\mu = \sum_{i=1}^{N} \frac{s_i}{N} \tag{3}$$

$$\hat{\mu} = \sum_{i=1}^{N} \frac{(s_i - \delta)}{N} = \left(\sum_{i=1}^{N} \frac{s_i}{N} \right) - \delta = \mu - \delta. \tag{4}$$

Similarly, the variance of S and $\hat{S}$ can be shown to be equal, regardless of δ.

$$\sigma^2 = \sum_{i=1}^{N} \frac{(s_i - \mu)^2}{N-1} \tag{5}$$

$$\begin{aligned} \hat{\sigma}^2 &= \sum_{i=1}^{N} \frac{(\hat{s}_i - \hat{\mu})^2}{N-1} \\ &= \sum_{i=1}^{N} \frac{(s_i - \delta - (\mu - \delta))^2}{N-1} \\ &= \sum_{i=1}^{N} \frac{(s_i - \mu)^2}{N-1} = \sigma^2. \end{aligned} \tag{6}$$

As such, basic statistical metrics are of little use applied to benchmark data, including both the FTQ and FWQ methods presented earlier. To truly extract non-trivial information from a time-series, more sophisticated techniques are required.

Finally, it is frequently the case that time-series data is distilled down to a single visualization that collapses the time axis out of the sequence. One such method seen in practice is the histogram. Fig. 9 shows that two signals with very different periodic structure yield the same histogram.

A periodic signal may in fact produce a visually interesting histogram, but the structure of the histogram easily can be shown to have little or no correlation with periodic structure in the source data. Furthermore, the histogram can in fact strip away all interesting information other than a visual representation of μ and σ. As an example, consider the zero-mean first order differenced data in histogram form shown in Fig. 5.

V. Conclusions and Ongoing Work

The work presented in this paper is an active research area. Other groups have used similar techniques based on the FWQ benchmark to parameterize models and simulations for identifying noise [8]. That work has been highly successful

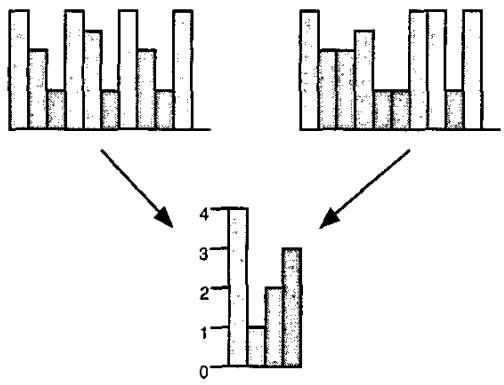


Fig. 9. Two different time-series with an identical histogram. Shading of histogram "buckets" corresponds to shading of samples.

and continues to provide significant insight about new and existing systems. Noticing the similarity between benchmark sampling and discrete signal processing, we have found that a carefully constructed benchmark opens up the world of DSP techniques to us for our analysis process.

In the process of this research, we were confronted by one of the performance analyst's most notorious difficulties - timer precision and accuracy. The high-precision Pentium TSC counter (and similar counters on other architectures) provides a viable solution for such simple benchmarks in identifying periodic noise that has a frequency orders of magnitude coarser than the jitter in the TSC timer. Given a high-precision timer, we were then able to create the FTQ benchmark - a tool for sampling the performance of the system in the time domain. With this measurement tool, we then began the process of exploring analysis techniques from signal processing and sampling theory to extract and identify features within the noise.

Our current work is focused on perfecting how we apply these techniques, and then interpreting the features that are extracted to correlate them with concrete hardware and software activities that are the ultimate source of noise and the resulting application performance degradation. A direct consequence of the sampling theorem is that due to the maximum sampling rate being dictated by the processor clock, there is a large portion of the spectrum at high-frequencies that is inaccessible due to aliasing or the slight but unavoidable error on sampling interval boundaries. This contributes to the subtle but present noise that appears as low magnitude frequency components in the Fourier decomposition shown in Fig. 8. Application of high- and band-pass filters, filterbanks, and multi-rate analysis techniques (such as polyphase decomposition) from the DSP community may assist in separating interesting components from sampling noise. Furthermore, exploration of self-contained, computationally efficient tools to analyze microbenchmark data will assist non-experts in tuning their own cluster nodes.

Our end goal is to have the ability to measure a system, process the data and, using well-known signal processing techniques, provide quantitative assessments as to the nature of the interference, its magnitude, and its probable impact – or lack thereof – on the performance of an application on a node.

REFERENCES

[1] George E. P. Box, Gwilym M. Jenkins, and Gregory C. Reinsel. *Time Series Analysis: Forecasting and Control, Third Edition*. Prentice Hall, 1994.

[2] Kris Buytaert. The openMosix HOWTO. *http://howto.ipng.be/openMosix-HOWTO*, 2002.

[3] R. P. Draves, B. N. Bershad, and A. F. Forin. Using microbenchmarks to evaluate system performance. In *Proceedings of the Third Workshop on Workstation Operating Systems*, pages 148–153, 1992.

[4] Erik Hendriks. BProc: The Beowulf Distributed Process Space. *16th Annual ACM International Conference on Supercomputing*, 2002.

[5] Larry McVoy and Carl Staelin. lmbench: Portable tools for performance analysis. In *Proceedings of the USENIX Annual Technical Conference*, 1996.

[6] Alan V. Oppenheim, Ronald W. Schager, and John R. Buck. *Discrete-Time Signal Processing, Second Edition*. Prentice Hall, 1999.

[7] Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab. *Signals and Systems, Second Edition*. Prentice Hall, 1997.

[8] Fabrizio Petrini, Darren Kerbyson, and Scott Pakin. The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q. In *ACM/IEEE SC2003*, Phoenix, Arizona, November 10–16 2003.

[9] Carl Staelin and Larry McVoy. mhz: Anatomy of a micro-benchmark. In *Proceedings of USENIX technical conference*, 1998.

View publication stats